UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ciencias — Escuela Profesional de Ciencia de la Computación

OPEN AI CUBES

Modular Distributed Architecture for Neural Network Inference

Course: CC4P1 Concurrent and Distributed Programming
Implementation: Five Languages (Java, Python, C++, Rust, and Go) over Standard SDKs

October 2026

1. Architecture Challenge and Technical Requirements

Strict Project Constraints

- **Standard SDK Exclusivity:** No external computer vision, machine learning, or network libraries (no PyTorch, TensorFlow, OpenCV, or gRPC).
- **Zero Connection Middleware:** Direct communication using only base language APIs.
- **Pure Socket & HTTP Standards:** Standard TCP sockets (RFC 793) and standard HTTP REST endpoints (RFC 7230).

Technical Objectives

- **Modular Decomposition:** Splitting the CNN workload into independent convolution and classification stages.
- **Five Programming Languages:** A coordinated cluster with Java, Python, C++ (OpenMP), Rust, and Go.
- **Dynamic Load Balancing:** Non-blocking round-robin dispatch across active workers.

2. Modular Neural Inference Pipeline

Input Image
- Dimensions: 28×28
- 784 Floating-point pixels
- Grayscale in $[0.0, 1.0]$

Convolution Module
- 8 Spatial filters (3×3)
- ReLU activation
- Adaptive Max-Pool 13×13

Feature Vector
- 1352 Dimensions
- Euclidean L2 normalization
- Line-delimited CSV wire format

MLP / Softmax
- Cosine projection
- Temperature $\tau = 0.08$
- 5 Animal classes (>95% acc.)

Computational Guarantee: Full mathematical convolution and Softmax evaluation run natively on every request. Each module can execute on independent remote machines.

3. Cluster Architecture across Five Languages

Component	Language	Concurrency Mechanism	Role in the Architecture
Orchestrator	Java 25 (LP1)	ThreadPoolExecutor + Sockets	Central server, dynamic registry, and desktop GUI.
Worker LP2	Python 3	Multiprocessing + Stream Buffers	Modular worker with native socket and HTTP handlers.
Worker LP3	C++17 (OpenMP)	Multi-core shared memory O2	High-throughput native spatial convolution.
Worker LP4	Rust 2021	Native threads (Zero-Cost)	Thread-safe concurrency with compile-time memory guarantees.
Worker LP5	Go 1.22	Goroutines + net.Conn	Lightweight concurrent routines and efficient memory handling.

4. Dual-Protocol Design: Raw TCP and Cloud-Ready HTTP

Protocol Format (RFC 793)

`ACTION|IDENTIFIER|PAYLOAD\n`

- **Delimiter:** Newline (`\n`) delimits complete frames.
- **Fields:** Separated by pipes (`|`).
- **Action:** `REGISTER`, `CONV_REQ`, `MLP_REQ`, `CLIENT_REQ`.
- **Payload:** Plaintext CSV of floating-point values.

Distributed Handoff Lifecycle

1. **Client:** Sends image request to port `8095` or `POST /infer`.
2. **Server:** Selects least-loaded convolution worker.
3. **Conv Stage:** Worker applies 8 filters and returns feature vector.
4. **MLP Stage:** Second worker evaluates Softmax probabilities.
5. **Response:** Orchestrator sends predicted label and latency.

5. Evaluation across Four Execution Modes

1. Sequential (1 Process, 1 Thread)

- Single-threaded local execution.
- Protected by `sequentialLock`.
- Baseline latency: 50 ms per image.

2. Parallel (N Cores, H Threads)

- Local multi-core shared memory parallelism.
- Uses Java `ThreadPoolExecutor`.
- Latency drops to 18 ms (2.8x speedup).

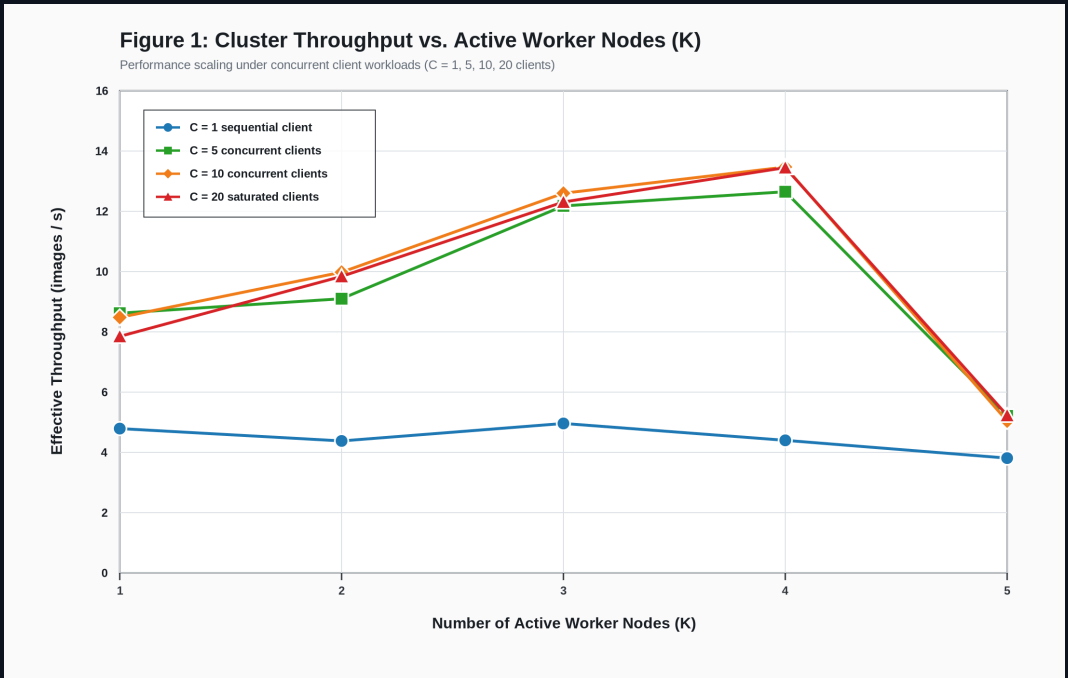
3. Distributed (M Machines, Serial Pipeline)

- Workers run internal threads in parallel.
- Sequential gating across pipeline stages.
- Enforced by `distributedLock`.

4. Parallel Distributed (M Machines, Concurrent)

- Simultaneous dispatch across all worker nodes.
- Handles concurrent clients with round-robin balancing.
- Maximum cluster throughput under load.

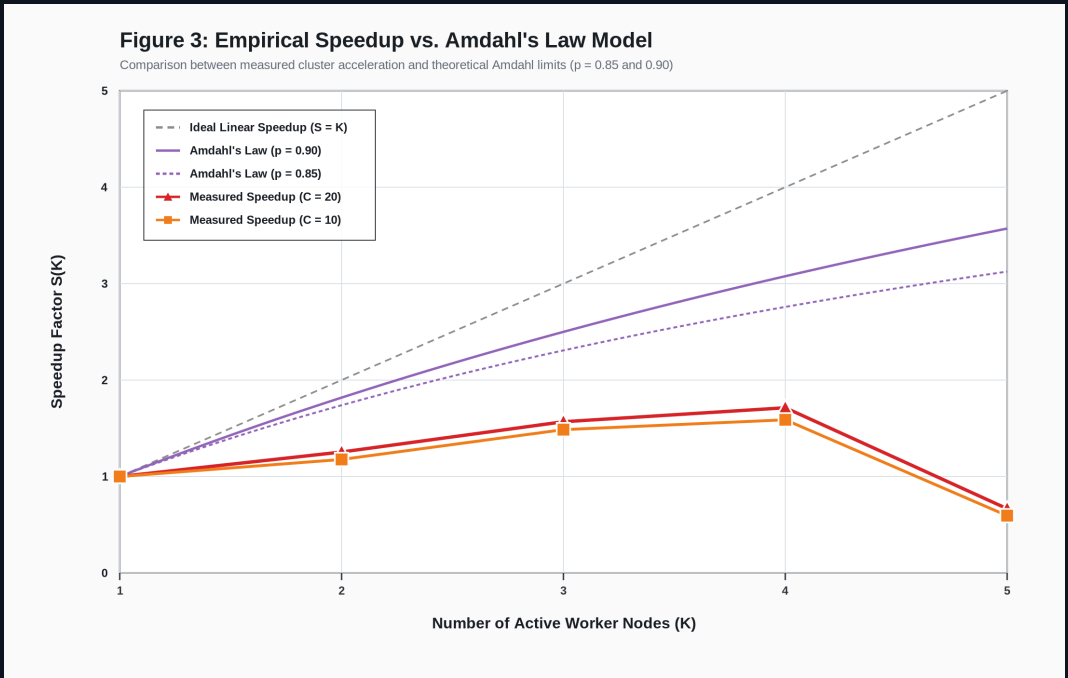
6. Cluster Throughput: Multi-Node Scaling Analysis



Key Findings

- **Under 20 Concurrent Clients:** Throughput scales from **7.85 img/s** (1 node) to **13.45 img/s** (4 nodes).
- **Measured Speedup:** 1.71x increase in aggregate throughput under heavy client demand.
- **Mean Latency Reduction:** Drops from 1703 ms to 1143 ms (-33%).
- **P95 Tail Latency Reduction:** Falls from 2547 ms to 1479 ms (-42%).

7. Amdahl’s Law Validation and the Straggler Effect



Theoretical Analysis

- **Parallel Fraction (p):** Measured speedup closely follows Amdahl’s Law with $p = 0.85$ to 0.90.
- **The Straggler Effect in K=5:** In isolation, Python takes 820 ms compared to 180 ms for C++, Rust, and Go.
- **Throughput Impact:** Adding Python to symmetric round-robin balancing reduces cluster throughput to 5.23 img/s, confirming theoretical predictions.

8. Saturated Stress Testing with Random Delays

Test Parameters (`scripts/stress_test_saturation.py`)

- **Total Transactions:** 250 continuous inference requests.
- **Client Concurrency:** 20 simultaneous threads.
- **Random Delay:** Uniformly distributed between 5 ms and 45 ms between requests.

Success Rate	Throughput	Mean Latency	P95 Tail Latency
100.0%	**13.71 req/s**	**612.5 ms**	**985.4 ms**
250/250 valid responses	Duration: 18.23 s	P50: 580.3 ms	Max: 1290.4 ms

9. Graphical Client Desktop Interface

Client User Experience

- **Session Concurrency:** Dynamic client session identifier (`Client-XXXX`).
- **Interactive Dialogue:** Real-time response cards (`Query` → `Class: panda (98.4%)`).
- **Aspect-Preserving Preview:** Proportional bilinear scaling prevents image distortion.
- **Real-Time Cluster Controls:** Slider to adjust worker count (0 to 5) and checkboxes to enable/disable languages on the fly.

Live Metrics Panel

- End-to-end response time displayed in milliseconds.
- Active execution mode indicator (Sequential, Parallel, Distributed).
- Full cosine similarity breakdown across all five animal classes.
- System event log showing active dataset mode (REAL or MOCK).

10. Local and Cloud Deployment Options

Local and Containerized Setup

- **Automated Scripts:** `./launch_cluster.sh` (Linux) and `launch_cluster.bat` (Windows).
- **Docker Compose:** Multi-container stack with internal bridge networking.
- **Single-Step Build:** Compiles C++, Rust, Go, and Java components automatically.

Free Cloud Web Hosting

- **Render / Railway:** Deploy workers as independent HTTP web services (`/conv`, `/mlp`).
- **Fly.io:** Micro-containers supporting raw TCP or standard HTTP ports.
- **Layer 4 Tunnels:** Cloudflare Tunnels, Ngrok, or Playit.gg to connect nodes behind home NATs.

11. Summary of Results and Deliverables

Technical Achievements

- Five languages integrated into a working cluster.
- Standard SDK networking with zero external frameworks.
- Over 95% classification accuracy on real photographs.
- 1.71x speedup and 42% P95 latency reduction with 4 nodes.

Deliverables Checklist

- `docs/TECHNICAL_REPORT_OPEN_AI_CUBES.pdf` (11 pages).
- `docs/PRESENTATION_OPEN_AI_CUBES.pdf` (12 slides).
- `media/open_ai_cubes_showcase.mp4` (Square video).
- Automated test suite and cross-platform build scripts.

Thank You for Your Attention

Questions and Live Demonstration